🎨 UI/UX Design: Crafting the User Experience

> [!NOTE] Welcome! In this document, we will look at the **UI/UX (User Interface and User Experience)** design of the Kiosk Frontend. This is the part of the system that the user actually sees, touches, and interacts with.

No matter how powerful the AI is behind the scenes, if the interface is confusing or boring, the project will fail. Our goal is to make the experience **intuitive**, **highly engaging**, and **visually stunning**.

🎯 The Goal

To guide users smoothly through a multi-step AI process without them feeling overwhelmed, while keeping them entertained during the few seconds it takes for the AI to generate their images.

⚙️ The Technical Stack

- **HTML:** Provides the structure (buttons, screens, camera feed).
- **CSS (Vanilla):** Handles all the styling, layouts, and beautiful animations. We didn't use heavy frameworks like React or Tailwind, keeping the application lightweight, fast, and highly customizable.
- **JavaScript:** Manages the logic (switching screens, talking to the webcam, polling the server).
- **Flask (Python):** The local backend that bridges the UI with the AI generation engine.

🌟 Core Design Principles

1. Thematic Visuals (The "Vibe")

Because this is a themed interactive booth, the colors and elements must reflect an immersive atmosphere:

- **Color Palette:** A vibrant, cohesive color palette designed to grab attention and reflect a premium feel.
- **Typography:** Using elegant, modern fonts to make the text highly readable and stylish.

2. Micro-Animations and "Alive" UI

A static screen is boring. We used CSS animations to make the interface feel alive even when the user isn't doing anything:

- **Floating Elements:** Background items, dynamic icons, and text gently float up and down on the screen using CSS keyframes, giving the interface a sense of continuous motion.
- **Button Hovers:** When a user hovers over a button, it scales up slightly and glows, providing immediate visual feedback that it is clickable.

3. Step-by-Step Flow (Wizard Pattern)

Asking a user to select their gender, pose, clothes, background, and take a photo all on one screen would be chaotic. Instead, we use a **"Wizard"** approach:

- The UI is broken into distinct **"Screens"** (`welcome-screen`, `pose-screen`, `camera-screen`, etc.).
- JavaScript hides all screens except the active one.
- This bite-sized progression keeps the user focused on one choice at a time.

4. Engaging the User During "Wait Times"

AI image and video generation takes time (sometimes 10 to 30 seconds). In UX design, making a user stare at a blank loading bar is a cardinal sin.

- **Interactive Loading Screens:** While the AI works, we transition to an active processing screen titled *"Creating Magic..."* featuring spinning glowing visuals and engaging text.
- **Fun Disclaimers:** We present humorous or interesting disclaimers to read while the models are running.

> [!TIP] Providing entertainment and visual stimuli makes the waiting period feel significantly shorter!

5. Seamless Hardware Integration

- **Webcam Access:** We use the browser's `navigator.mediaDevices` API to pull the live video feed directly into an HTML `<video>` tag.
- **Face Guide:** We overlay a translucent "frame" on the camera screen telling the user *"Please stand in the center,"* ensuring the AI gets a perfect, well-aligned photo every time.

🌐 6. The Network Pipeline: A Two-Stage Architecture

Behind the beautiful UI is a robust network pipeline. It is broken into two distinct stages to ensure maximum performance:

1. 🏠 **Stage 1: Local Image Generation:** Once the photo is taken, the JavaScript sends an HTTP POST request to the **local Flask server** (`app.py`). The local server handles the immediate Image Generation (using ComfyUI) and returns the final picture to the kiosk screen so the user sees results quickly.
2. 🚀 **Stage 2: Remote Video Generation:** If the user loves the photo, they click "Print & Animate". The kiosk then fires *another* POST request (`/api/transfer`), transferring the original and generated images to the **powerful Linux DGX Spark server** (the Receiver) for the heavy FLF video generation.
3. ☁️ **Outputting to Cloudflare:** Once the Linux server finishes generating the video, it uses a Cloudflare Tunnel. This tunnel creates a secure, temporary public URL pointing directly to the DGX backend server.
4. 📱 **Mobile Delivery:** As soon as the video generation is complete, the DGX backend generates a unique QR code linked to that Cloudflare URL. The UI detects completion and displays the QR code, letting users easily scan and download their AI-generated masterpieces.

💻 Technical Deep Dive: Code Snippets

Here is a closer look at the actual code that powers the different moving parts of our interface and network pipeline:

📸 1. Seamless Hardware Integration (Webcam)

To access the kiosk's camera directly from the web browser without any extra software, we use the standard WebRTC API:

```
// Accessing the webcam video stream
const startCamera = async () => {
    try {
        const stream = await navigator.mediaDevices.getUserMedia({
            video: { facingMode: "user" }
        });
        const videoElement = document.getElementById('camera-feed');
        videoElement.srcObject = stream;
    } catch (err) {
        console.error("Camera access denied or unavailable", err);
    }
};
```

🔀 2. Step-by-Step Flow (Screen Switching)

The "Wizard" approach works by having all screens defined in the HTML, but only showing one at a time using a simple CSS class toggle:

```
// Function to switch between UI screens
function showScreen(screenId) {
    // Hide all screens
    document.querySelectorAll('.screen').forEach(screen => {
        screen.classList.remove('active');
    });
    // Show the target screen
    document.getElementById(screenId).classList.add('active');
}
```

🪄 3. Micro-Animations (Floating Elements)

To keep the UI alive without slowing down JavaScript, we use pure CSS keyframes. The browser's graphics card handles these animations effortlessly:

```
/* Define the floating animation */
@keyframes float {
    0%   { transform: translateY(0px); }
    50%  { transform: translateY(-15px); }
    100% { transform: translateY(0px); }
}

/* Apply it to an element */
.floating-element {
    animation: float 4s ease-in-out infinite;
}
```

⏱️ 4. Wait Time Entertainment (Live Polling & Analytics)

Because AI generation takes time, the frontend uses `setInterval` to repeatedly "poll" the local Flask backend. It retrieves both the job status and the live facial analytics to keep the user entertained!

```
// Checking if the AI has finished and updating live text
function pollStatus(jobId) {
    const pollInterval = setInterval(async () => {
        const status = await checkStatus(jobId);

        // Update the loading screen with live facial analytics
        if (status.analytics) {
            updateAnalytics(status.analytics);
        }

        if (status.status === 'completed') {
            clearInterval(pollInterval);
            displaySuccessImage(status.image); // Reveal the final image!
        }
    }, 2000); // Check every 2 seconds
}
```

📡 5. The Network Pipeline (The Two-Stage Transfer)

The first request creates the static image locally. The second request transfers it to the massive DGX server for animation.

```
// STAGE 1: Send photo to LOCAL Flask Server for immediate image
generation
const response = await fetch('/api/process', {
    method: 'POST',
    body: formData
});

// STAGE 2: If user likes the image, trigger transfer to remote DGX Server
for animation!
document.getElementById('animate-btn').addEventListener('click', async ()
=> {
    await fetch(`/api/transfer/${state.currentJobId}`, { method: 'POST'
});
});
```

🔗 6. Outputting to Cloudflare (Mobile Delivery)

Once the files are ready, the backend generates a QR code. The QR code points directly to the secure Cloudflare tunnel URL where the user can view and save their result.

```python
# Python Backend snippet: Generating the QR Code for the Cloudflare URL
import qrcode
import io

@app.route('/api/qr/<job_id>')
def get_qr_code(job_id):
    # The Cloudflare URL dynamically maps to our local server
    public_url = "https://example-tunnel.trycloudflare.com"
    mobile_view_url = f"{public_url}/mobile/view/{job_id}"

    # Generate the QR Code image
    qr = qrcode.make(mobile_view_url)

    # Send image back to the Kiosk screen
    img_buffer = io.BytesIO()
    qr.save(img_buffer, format='PNG')
    img_buffer.seek(0)

    return send_file(img_buffer, mimetype='image/png')
```